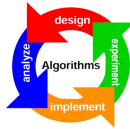


Estrategias algorítmicas

José A. Rodríguez Melquiades

21 de junio de 2026

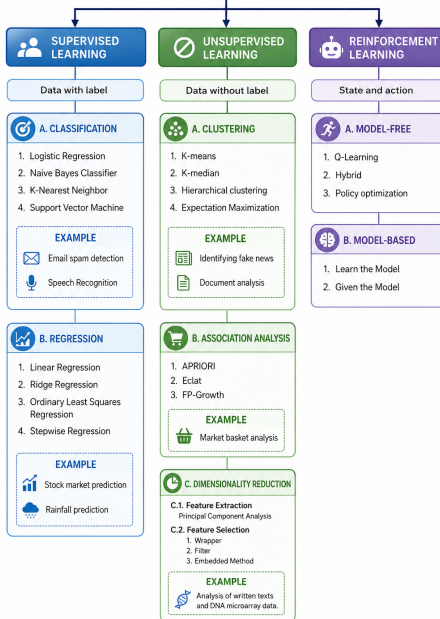
Algoritmos dividir para vencer



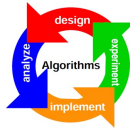
- 1 Aprendizaje automático
- 2 Estrategia de Localización-Asignación para sectorizar
- 3 Detección de tumores mamarios
- 4 VRP con Dividir para conquistar (KMeans + TSP Greedy)



MACHINE LEARNING



Algoritmos dividir para vencer



- 1 Aprendizaje automático
- 2 Estrategia de Localización-Asignación para sectorizar
- 3 Detección de tumores mamarios
- 4 VRP con Dividir para conquistar (KMeans + TSP Greedy)

Estrategia de Localización-Asignación para sectorizar

- **Idea:** Dividir una área geográfica en regiones (sectores optimizados considerando la ubicación de puntos de interés o clientes), mediante KMeans y luego asignar clientes a centros que pueden proporcionar algún servicio de salud, logística, etc.

Estrategia de Localización-Asignación para sectorizar

- **Idea:** Dividir una área geográfica en regiones (sectores optimizados considerando la ubicación de puntos de interés o clientes), mediante KMeans y luego asignar clientes a centros que pueden proporcionar algún servicio de salud, logística, etc.
- **Importancia:** En aplicaciones como: Planificación urbana, logística de distribución y la gestión de servicios, donde la eficiencia en la asignación de recursos y la optimización de la cobertura geográfica son fundamentales para mejorar la calidad del servicio y reducir costos operativos.

- Supuestos:

- **Supuestos:**

- ▶ **Cientes:** Localizados, coordenadas geográficas, en cierta zona.

- **Supuestos:**

- ▶ **Clientes:** Localizados, coordenadas geográficas, en cierta zona.
- ▶ **Problema:** Sectorizar el área o zona geográfica en k regiones.

- **Supuestos:**

- ▶ **Clientes:** Localizados, coordenadas geográficas, en cierta zona.
- ▶ **Problema:** Sectorizar el área o zona geográfica en k regiones.
- ▶ **Proceso.**

- **Supuestos:**

- ▶ **Clientes:** Localizados, coordenadas geográficas, en cierta zona.
- ▶ **Problema:** Sectorizar el área o zona geográfica en k regiones.
- ▶ **Proceso.**
 - ★ **Seleccionar:** k centros de servicio (location).

- **Supuestos:**

- ▶ **Clientes:** Localizados, coordenadas geográficas, en cierta zona.
- ▶ **Problema:** Sectorizar el área o zona geográfica en k regiones.
- ▶ **Proceso.**
 - ★ **Seleccionar:** k centros de servicio (location).
 - ★ **Asignar:** Clientes a su centro más cercano (allocation).

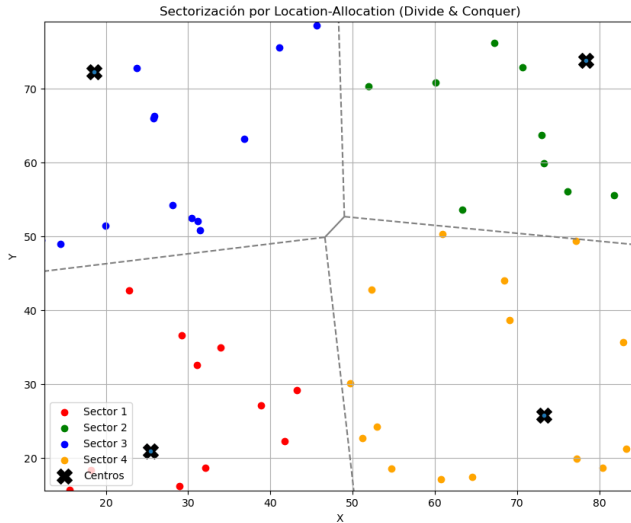
- **Supuestos:**

- ▶ **Clientes:** Localizados, coordenadas geográficas, en cierta zona.
- ▶ **Problema:** Sectorizar el área o zona geográfica en k regiones.
- ▶ **Proceso.**
 - ★ **Seleccionar:** k centros de servicio (location).
 - ★ **Asignar:** Clientes a su centro más cercano (allocation).

- **Estrategia:** Dividir para conquistar a través de KMeans (dividir) y luego Voronoi/Centro cercano (conquistar).

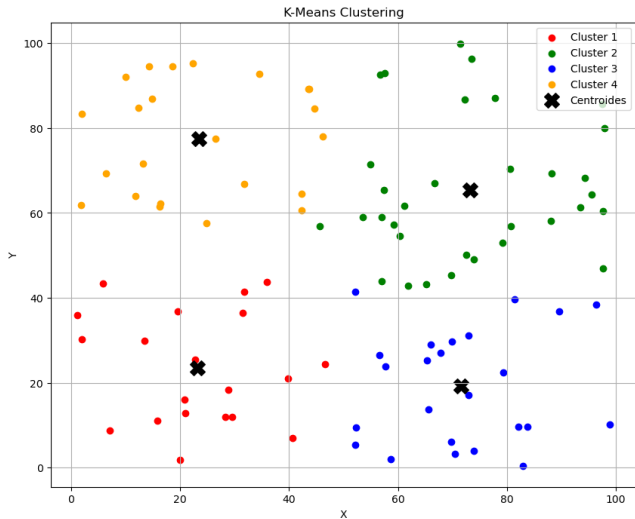
● Seudocódigo: Localización-Asignación

- 1: **Entrada:** NumCliente; CoordClientes; k ▷ k: número de sectores
- 2: **Salida:** Coord de cada centro; Cantidad de clientes asignados por sector.
- 3: Aplicar KMeans a los clientes con k sectores ▷ Dividir en sectores
- 4: Retornar: Etiquetar el sector para cada cliente; Coord de los centros del sector
- 5: Almacenar los clientes por sector. ▷ Conquistar: asignar clientes a centros
- 6: **for** Cada cliente **do**
- 7: Etiquetar el cluster para signarlo al correspondiente sector
- 8: **end for**

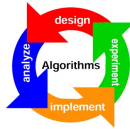


● Seudocódigo: KMeans

- 1: **Entrada:** D , k , $MaxIter$; Tol ▷ Conjunto de datos con N puntos; clústers deseados; umbral mínimo de cambio en centroides para detenerse
- 2: **Salida:** Lista con k centroides; asignación de cada punto a un cluster
- 3: Inicializar k centroides de forma aleatoria desde D
- 4: **repeat**
- 5: **for** $x \in D$ **do** ▷ Asignación
- 6: Calcular la distancia de x a cada uno de los k centroides
- 7: Asignar x al clúster del centroide más cercano
- 8: **end for**
- 9: **for** cada clúster $i = 1 : k$ **do** ▷ Actualización
- 10: Hallar nuevo centroide ▷ el promedio de los pto. asignados al clúster
- 11: **end for**
- 12: **if** los nuevos centroides son muy similares a los anteriores **then**
- 13: Parar pues se alcanzo la convergencia
- 14: **end if**
- 15: **until** que los centroides no cambien o se alcanzo $MaxIter$



Algoritmos dividir para vencer



- 1 Aprendizaje automático
- 2 Estrategia de Localización-Asignación para sectorizar
- 3 Detección de tumores mamarios
- 4 VRP con Dividir para conquistar (KMeans + TSP Greedy)

Detección de tumores mamarios

- **Cáncer de mama:** El más frecuente y plantea un reto de salud pública para las mujeres de todo el mundo.

Detección de tumores mamarios

- **Cáncer de mama:** El más frecuente y plantea un reto de salud pública para las mujeres de todo el mundo.
- **Tratamiento:** Debe ser eficaz y depende de los mecanismos de diagnóstico precoz considerados.

Detección de tumores mamarios

- **Cáncer de mama:** El más frecuente y plantea un reto de salud pública para las mujeres de todo el mundo.
- **Tratamiento:** Debe ser eficaz y depende de los mecanismos de diagnóstico precoz considerados.
- **Mamografía:** Es un método eficaz para detectar el cáncer de mama de manera oportuna.

Detección de tumores mamarios

- **Cáncer de mama:** El más frecuente y plantea un reto de salud pública para las mujeres de todo el mundo.
- **Tratamiento:** Debe ser eficaz y depende de los mecanismos de diagnóstico precoz considerados.
- **Mamografía:** Es un método eficaz para detectar el cáncer de mama de manera oportuna.
- Estudiamos el uso de la estrategia dividir para vencer aplicado a la detección de tumores en la mama.

- Estrategia computacional:

- **Estrategia computacional:**

- ▶ **Dividir:** Recorrer y subdividir la imagen en bloques. Cada bloque aparecerá en una visualización secuencial de color amarillo.

- **Estrategia computacional:**

- ▶ **Dividir:** Recorrer y subdividir la imagen en bloques. Cada bloque aparecerá en una visualización secuencial de color amarillo.
- ▶ **Conquistar:** Cada bloque es analizado y marcado con color azul si es sospechoso de contener un tumor. Caso contrario se marca de color verde.

- **Estrategia computacional:**

- ▶ **Dividir:** Recorrer y subdividir la imagen en bloques. Cada bloque aparecerá en una visualización secuencial de color amarillo.
- ▶ **Conquistar:** Cada bloque es analizado y marcado con color azul si es sospechoso de contener un tumor. Caso contrario se marca de color verde.
- ▶ **Combinar:** Se muestra una imagen unificada con todos los bloques sospechosos destacados.

- **Seudocódigo:** Dividir para conquistar para detección de tumores

```
1: Entrada: Rutalmagen, WindowSize, threshold    ▷ threshold = límite o umbral
2: Salida: visualización y resultado final con zonas sospechosas
3: Imagen:=
4: function CARGARIMAGEN(Rutalmagen)                ▷ Imagen en escala gris
5: end function
6: function FASEDIVIDIR(Imagen, WindowSize)
7:   for cada bloque (x, y) de tamaño WindowSize en la imagen do
8:     Dibujar un rectángulo amarillo en la región actual
9:     Mostrar el bloque actual para representar la división
10:  end for
11: end function
```

- Seudocódigo: (cont.)

```
1: RegionesSospechosas:=
2: function FASECONQUISTAR(Imagen, WindowSize, threshold)
3:   Inicializar lista vacía de RegionesSospechosas
4:   for cada bloque  $(x, y)$  de tamaño WindowSize en la imagen do
5:     subimagen:= extraer región  $(x, y)$  de tamaño WindowSize
6:     if PromedioIntensidad(subimagen) > threshold then
7:       Agregar  $(x, y)$  a lista de RegionesSospechosas
8:       Dibujar un rectángulo azul en la imagen
9:     else
10:      Dibujar un rectángulo verde en la imagen
11:    end if
12:    Mostrar el bloque procesado
13:    Visualizar resultado
14:  end for
15: end function
```

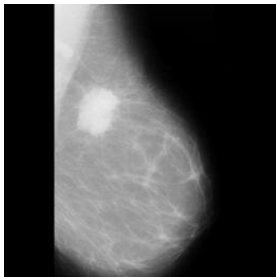
● Seudocódigo: (cont.)

```
1: function FASECOMBINAR(Imagen, RegionesSospechosas, WindowSize)
2:   for cada región  $(x, y)$  en RegionesSospechosas do
3:     Dibujar un rectángulo azul sobre la región
4:   end for
5:   Mostrar la imagen final con todas las regiones marcadas
6: end function
```

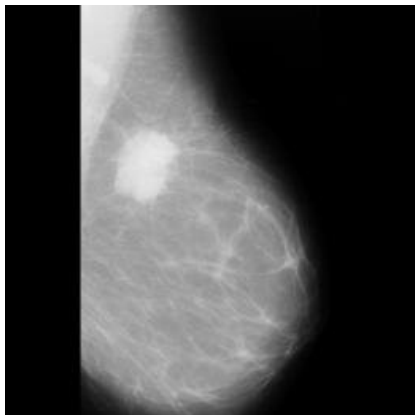
- Seudocódigo: (cont.)

```
1: function FASECOMBINAR(Imagen, RegionesSospechosas, WindowSize)
2:   for cada región (x, y) en RegionesSospechosas do
3:     Dibujar un rectángulo azul sobre la región
4:   end for
5:   Mostrar la imagen final con todas las regiones marcadas
6: end function
```

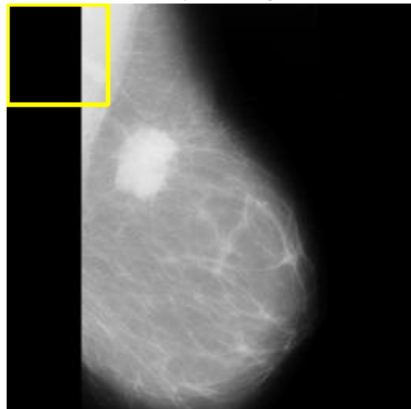
- Imágenes:



División: Explorando subregiones



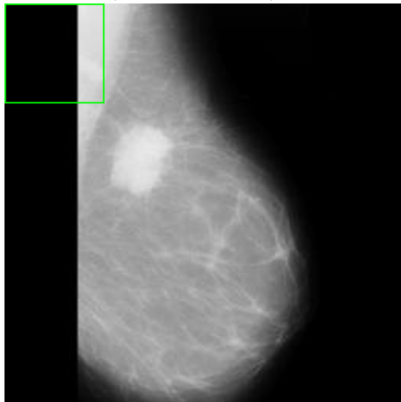
(a) Imagen original



(b) División en subproblemas

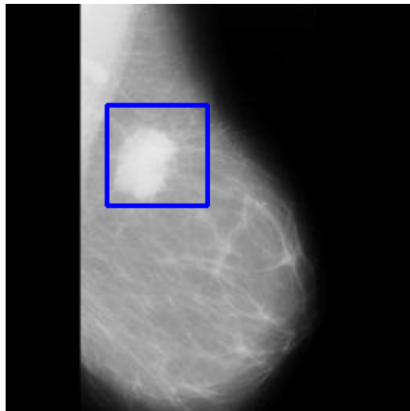
Figura 1: Proceso dividir para vencer

Conquista: Detectando zonas sospechosas



(a) Conquistar los subproblemas

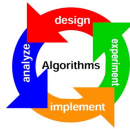
Combinación: Resultado Final



(b) Combinar de los subproblemas

Figura 2: Proceso dividir para vencer

Algoritmos dividir para vencer



- 1 Aprendizaje automático
- 2 Estrategia de Localización-Asignación para sectorizar
- 3 Detección de tumores mamarios
- 4 VRP con Dividir para conquistar (KMeans + TSP Greedy)

VRP con Dividir para conquistar (KMeans + TSP Greedy)

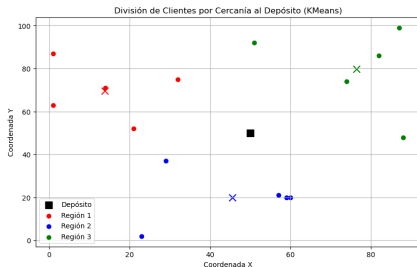
- **Dividir:** El conjunto de clientes en regiones cerca al depósito usando KMeans

VRP con Dividir para conquistar (KMeans + TSP Greedy)

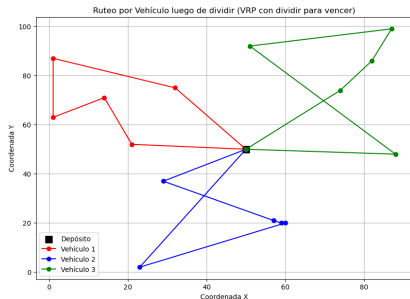
- **Dividir:** El conjunto de clientes en regiones cerca al depósito usando KMeans
- **Conquistar:** Resolver un TSP para cada ruta de cada vehículo.

VRP con Dividir para conquistar (KMeans + TSP Greedy)

- **Dividir:** El conjunto de clientes en regiones cerca al depósito usando KMeans
- **Conquistar:** Resolver un TSP para cada ruta de cada vehículo.
- **Combinar:** Unir los recorridos.



(e) Dividir en subproblemas



(f) Combinar de los subproblemas

● Seudocódigo:

```
1: Entrada:  $n\_Clientes$ ,  $n\_Vehiculos$ 
2: Salida:  $VRP$ 
3: Crear un depósito
4: Generar localizaciones aleatorias para  $n\_Clientes$ 
5: Aplicar KMeans con  $n\_clusters = n\_Vehiculos$ 
6: for vehículo  $v = 1 : n\_Vehiculos$  do
7:     Obtener clientes del clúster
8:     Unir depósito con puntos del clúster
9:     Inicializar ruta con depósito:  $Recorrido := [0]$ 
10:    Inicializar no visitados:  $NoVisitado := \{1, \dots, n\}$ 
11:    while  $NoVisitado \neq \emptyset$  do
12:         $last :=$  último nodo en  $Recorrido$ 
13:         $next :=$  nodo más cercano a  $last$  en  $NoVisitado$ 
14:        Agregar  $next$  a  $Recorrido$ 
15:        Remover  $next$  de  $NoVisitado$ 
16:    end while
17:    Imprimir y visualizar ruta  $Recorrido$ 
18: end for
```